Testing and Quality Assurance Report

Course Code: CSE-416

Course Title: Software Project Design and Development

Submitted to: Dr. Mahfida Amjad Dipa

Group Members: Zannatul Nayem (0432320005101063)
Reon Ahmed (0432320005101066)

Section: 7B

Batch: 54

Semester: Autumn 2026

Project: Automated MCQ Exam Portal

Testing focus: backend unit testing, integration testing, boundary/edge-case analysis, defect correction, role-based authentication validation, and coverage measurement using Jest and NestJS testing utilities.

Prepared from the verified local test run completed in November 2026.

Software Testing and Quality Assurance

Automated MCQ Exam Portal

Project type	Full-stack role-based online examination platform
Test framework	Jest + ts-jest for the NestJS/TypeScript backend
Tool-assisted review	Qodo used as a supplementary AI-assisted review/quality tool on the pull request
Final execution	9 test suites passed; 46 tests passed

1. Objective

Design and execute a testing strategy for the Automated MCQ Exam Portal, verify core business rules, exercise important controller/service flows, expand boundary and negative cases, record coverage, and fix defects found during testing.

2. Testing Strategy

Step	Activity	Application to this project
1. Design	Create unit and integration tests using equivalence partitions and boundary values.	Authentication, authorization, exam rules, question validation, evaluation, and exam API flows.
2. Refine	Add edge-case tests and inspect tool-assisted review feedback.	Password boundaries, passing-mark limits, question-pool limits, invalid options, role mismatch, negative-marking cases.
3. Deepen	Explore defects and coverage gaps that normal-path tests may miss.	Unanswered questions, invalid scheduling, unauthenticated access, multi-role access, invalid business-rule inputs.
4. Verify	Fix revealed defects and rerun the complete suite.	Final result: 9/9 suites and 46/46 tests passed.

3. Tool-Assisted Testing / Qodo

Qodo was used as a supplementary AI-assisted review tool on the project pull request. Its review was used to inspect the submitted changes and look for issues. The executable test suite remained under Jest, and the final test assertions were manually validated and executed before the final result was recorded.

The project did not use Diffblue. Because this project is implemented in NestJS/TypeScript, Diffblue was not treated as an applicable testing tool here.

4. Representative Test Case Report

Representative cases from the implemented automated suite are listed below. The complete suite contains additional assertions and scenarios.

ID	Scenario	Input / condition	Expected result	Status
TC_01	Valid login DTO	Valid email, password, STUDENT role	Validation succeeds	**PASS**
TC_02	Invalid email	Malformed email	Email validation error	**PASS**
TC_03	Password lower boundary	8-character registration password	Validation succeeds	**PASS**
TC_04	Password below boundary	7-character registration password	Password validation error	**PASS**
TC_05	Role match	Required TEACHER; user=TEACHER	Access allowed	**PASS**
TC_06	Role mismatch	Required ADMIN; user=STUDENT	Access denied	**PASS**
TC_07	No authenticated user	Protected role required; user undefined	Access denied	**PASS**
TC_08	Passing mark lower	passingMark=0	Validation succeeds	**PASS**
TC_09	Passing mark upper	passingMark=100	Validation succeeds	**PASS**
TC_10	Passing mark below	passingMark=-0.01	Validation error	**PASS**
TC_11	Passing mark above	passingMark=100.01	Validation error	**PASS**
TC_12	Minimum duration	durationMinutes=0	Validation error	**PASS**
TC_13	Question pool exact	Attempt count equals pool size	Exam creation succeeds	**PASS**
TC_14	Question pool overflow	Attempt count exceeds pool	BadRequestException	**PASS**
TC_15	Too few options	Question has <2 options	BadRequestException	**PASS**
TC_16	No correct option	All options false	BadRequestException	**PASS**
TC_17	Multiple correct options	>1 option marked correct	BadRequestException	**PASS**
TC_18	Negative marking	2 correct, 1 wrong, 1 unanswered	1.75 score; 43.75%; fail	**PASS**
TC_19	Passing threshold	Percentage meets passing mark	passed=true	**PASS**
TC_20	Invalid schedule	End time before start time	BadRequestException	**PASS**

ID	Scenario	Input / condition	Expected result	Status
TC_21	Create-exam integration	Controller + service + Prisma mock	Exam persisted; audit log called	**PASS**
TC_22	Available-exam integration	Student courses passed into lookup	Published exams returned	**PASS**

5. Execution Results

9	46	0
Test suites passed	Tests passed	Test failures

6. Coverage Snapshot

The captured project coverage report recorded the following overall snapshot. These values describe the generated coverage report and should not be read as complete coverage of every application module.

Metric	Covered	Total	Coverage
Statements	210	874	24.02%
Branches	45	215	20.93%
Functions	26	158	16.45%
Lines	175	740	23.64%

7. Defects Found and Fixed

Defect	Observation	Resolution	Status
Evaluation test syntax/brace issue	A test file initially failed to parse because of a missing closing brace.	Corrected the test structure and reran the suite.	**Fixed**
Evaluation boundary expectation	One evaluation assertion initially disagreed with the intended business result.	Adjusted the assertion and reran the evaluation suite.	**Fixed**
Authentication boundary coverage	Boundary/invalid-input cases were expanded during QA refinement.	Added valid/invalid email and 7/8-character password cases.	**Verified**

8. Final Verification

The final automated test run completed with 9/9 test suites and 46/46 tests passing, and the backend production build completed successfully. The testing workflow concluded with a stable executable suite after defects were corrected and retested.

9. Test Suite Coverage by Module

Area	Implemented automated coverage
Authentication	Login/register DTO validation; valid/invalid email; password boundary cases
Authorization	Role match, mismatch, unauthenticated access, multi-role access
Exam management	Passing-mark boundaries, duration boundary, question-pool limits, schedule validation
Questions	Minimum options, zero correct option, multiple correct options, valid question creation
Evaluation	Negative marking, unanswered questions, percentage, pass/fail threshold
Exam API integration	Exam creation through controller/service; student course selection into available-exam lookup

10. QA Conclusion

The testing work followed the required sequence of designing tests, refining edge cases, using tool-assisted review, checking coverage, fixing defects, and rerunning the suite. The final result was 46/46 tests passing across 9 suites.

The report records the tools actually used. Diffblue was not used because this is a NestJS/TypeScript project.